

ข้อ 1

1.1 ข้อมูลนำเข้า (Input)

ข้อมูลนำเข้าของระบบประกอบด้วย:

ข้อมูลคลังสินค้า : จำนวน Stack ทั้งหมด (k), จำนวนกล่องทั้งหมดในระบบ (n), และความจุสูงสุดของแต่ละ Stack

ข้อมูลกล่อง (Box Data): ประกอบด้วย Box ID, Category, Arrival Time และ Priority

คำสั่งเป้าหมาย: รหัสกล่องเป้าหมาย (Target Box) ที่ต้องการนำออกจากคลังสินค้า

1.2 ผลลัพธ์ที่ต้องการ (Output)

นำกล่องเป้าหมายออกจาก Stack ได้สำเร็จ

กล่องอื่นๆ ที่ถูกย้ายเพื่อเปิดทาง ถูกนำกลับมาจัดเก็บอย่างถูกต้อง

1.3 ข้อจำกัดของปัญหา (Constraints)

การเข้าถึงข้อมูลต้องเป็นไปตามคุณสมบัติของ Stack (LIFO - Last In, First Out) กล่าวคือเข้าถึงได้เฉพาะกล่องบนสุด

ไม่สามารถย้ายกล่องไปยัง Stack ที่มีพื้นที่เต็มแล้วได้

1.4 ข้อตกลงที่ใช้ (Assumptions)

ในกรณีที่จำเป็นต้องย้ายกล่อง ระบบมีหน่วยความจำหรือตัวแปรชั่วคราวเพื่อจดจำลำดับของกล่องและ Stack ต้นทาง เพื่อให้สามารถย้ายกลับได้อย่างถูกต้องตามลำดับเดิม

1.5 กรณีขอบเขต (Edge Cases) และการจัดการข้อผิดพลาด

ระบบจำเป็นต้องรองรับและจัดการกรณีต่างๆ ดังนี้:

กรณีปกติ: กล่องเป้าหมายอยู่ตรงกลางของ Stack

กรณีขอบเขต (Edge Cases): กล่องเป้าหมายอยู่ด้านบนสุด หรือกล่องเป้าหมายอยู่ด้านล่างสุดของ Stack

กรณีที่อาจทำให้โปรแกรมผิดพลาด: ไม่พบกล่องเป้าหมายในระบบ, Stack ปลายทางบางกองเต็ม, ไม่มีพื้นที่เพียงพอสำหรับย้ายกล่อง หรือมี Stack ว่าง

ข้อ 2 Algorithm Designer (Method A) | • ออกแบบและเขียน Pseudocode ของ Algorithm วิธีที่ A

1. แนวคิดของ Algorithm

Algorithm A ใช้วิธี **First Available Stack** ในการย้ายกล่องที่อยู่ด้านบนและขวางกล่องเป้าหมาย

เมื่อพบว่ามีกล่องขวางอยู่ จะค้นหา Stack อื่นตั้งแต่กองแรกไปจนถึงกองสุดท้าย และเลือก **Stack แรก** ที่ยังมีพื้นที่ว่าง จากนั้นย้ายกล่องไปเก็บไว้ใน Stack นั้น

หากไม่พบ Stack ที่มีพื้นที่ว่างเพียงพอ จะทำการ **Rollback** กล่องที่ย้ายไปแล้วกลับคืน และยกเลิกการนำกล่องเป้าหมายออก

2. Pseudocode

ALGORITHM FirstAvailable(stacks, target)

1. Find sourceStack containing target

2. IF sourceStack is null THEN

 RETURN false

END IF

3. WHILE sourceStack.peek() $\neq$ target

4. box $\leftarrow$ Pop(sourceStack)

5. Find first Stack with available space

6. IF no Stack available THEN

7. Rollback moved boxes in reverse order

8. RETURN false

9. END IF

10. Push box into selected Stack

11. Record moved box

12. END WHILE

13. Pop target

14. Restore moved boxes in reverse order

15. RETURN true

3. Slide — Step-by-step Trace

Initial

- Stack 0: C $\rightarrow$ B $\rightarrow$ A (Target) โดย C อยู่บนสุด
- Stack 1:ว่าง
- Stack 2:ว่าง

Move C

- Stack 0: B $\rightarrow$ A (Target)
- Stack 1: C

- Stack 2: ว่าง

Move B

- Stack 0: A (Target)
- Stack 1: B $\rightarrow$ C
- Stack 2: ว่าง

Retrieve A

- Pop A ออกสำเร็จ
- Stack 0: ว่าง

Restore

- Pop B จาก Stack 1 $\rightarrow$ Push B กลับ Stack 0
- Pop C จาก Stack 1 $\rightarrow$ Push C กลับ Stack 0

Final

- Stack 0: C $\rightarrow$ B
- Stack 1: ว่าง
- Stack 2: ว่าง

ผลลัพธ์:

Box A ถูกนำออกสำเร็จ และ Box C กับ Box B กลับมาอยู่ในลำดับเดิม

4. กฎการเลือก Stack — First Available Selection

1. ตรวจสอบ Stack ตั้งแต่ดัชนี 0 ถึง k-1
2. ไม่เลือก sourceStack
3. ตรวจสอบว่า Stack มีพื้นที่ว่างหรือไม่
4. หากมีพื้นที่ว่าง ให้เลือก Stack นั้นทันที
5. ใช้ BREAK เพื่อหยุดการค้นหา
6. หากไม่มี Stack ใดมีพื้นที่ว่าง ให้ทำ **Rollback**

ตัวอย่าง

Stack 0 $\rightarrow$ ข้าม (เป็น Source Stack)

Stack 1 $\rightarrow$ เลือก (มีพื้นที่ว่าง)

Stack 2 $\rightarrow$ ไม่ต้องตรวจสอบต่อ

5. Restore และ Rollback

Restore

เมื่อสามารถนำ Target ออกได้สำเร็จ ระบบจะนำกล่องที่ถูกย้ายไปยัง Stack อื่นกลับมายัง Source Stack โดยใช้ข้อมูลจาก tempMovedList และนำกลับในลำดับย้อนกลับ (LIFO) เพื่อรักษาลำดับเดิมของกล่อง

Rollback

หากไม่สามารถหา Stack ปลายทางที่มีพื้นที่ว่างได้ ระบบจะนำกล่องที่ย้ายไปแล้วกลับคืน Source Stack ก่อน แล้วจึงคืนค่า false

- ข้อ 3 | Algorithm Designer (Method B) | • ออกแบบและเขียน Pseudocode ของ Algorithm วิธีที่ B
- เปรียบเทียบแนวคิดหลัก ข้อดี/ข้อจำกัดระหว่างวิธี A และ B
 - ช่วยเขียนอธิบายการทำงานในส่วนของวิธี B |

PseudocodeB

```
ALGORITHM RemoveCarRecursive(parkingStack, targetLicense)
INPUT:
    parkingStack : Stack ของรถที่จอดอยู่
    targetLicense : ทะเบียนรถเป้าหมายที่ต้องการนำออก
OUTPUT:
    boolean : true หากพบและนำรถออกสำเร็จ, false หากไม่พบรถ

1. IF parkingStack is empty THEN
2.     RETURN false
3. END IF
4. currentCar ← parkingStack.pop()
5. IF currentCar.licensePlate == targetLicense THEN
6.     RETURN true
7. END IF
8. found ← RemoveCarRecursive(parkingStack, targetLicense)
9. parkingStack.push(currentCar)
10. RETURN found
```

เปรียบเทียบแนวคิดหลักและข้อดี/ข้อจำกัดระหว่างวิธี A และ B

หัวข้อการเปรียบเทียบ	Method A: Iterative (คนที่ 2)	Method B: Recursive (คนที่ 3)
แนวคิดหลัก	ใช้ temporaryStack แบบ Explicit เพื่อพักรถที่ขวางทาง	ใช้ Call Stack ของระบบ (Implicit Stack) พักข้อมูล

		ตัวแปรชั่วคราวขณะย้อนกลับ
Best-case Time	$O(1)$ (เจอรถเป้าหมายที่ Top ทันที)	$O(1)$ (เจอรถเป้าหมายที่ Top ทันที)
Worst-case Time	$O(n)$ (รถอยู่ Bottom หรือไม่พบรถ)	$O(n)$ (รถอยู่ Bottom หรือไม่พบรถ)
Auxiliary Space	$O(n)$ (ใช้พื้นที่สำหรับ temporaryStack)	$O(n)$ (ใช้พื้นที่ Call Stack ตามความลึกของ Recursion)
ข้อดี	โค้ดตรงไปตรงมา ปลอดภัยจากการเกิด Stack Overflow	โค้ดสั้น กระชับ ไม่ต้องประกาศ Structure แยกสำรองเพิ่ม
ข้อจำกัด	ต้องเขียน Loop เพิ่มอีก 1 ลูปเพื่อคืนค่าข้อมูล	เสี่ยงต่อการเกิด StackOverflowError หากมีปริมาณรถจำนวนมาก (n สูง)

การทำงานของวิธี B (Recursive Method) มีขั้นตอนการทำงานตามลำดับของ Call Stack ดังนี้:

- 1. ตรวจสอบเงื่อนไขจุดหยุด (Base Case Check):**
 โปรแกรมจะตรวจสอบสถานะของ Stack หากพบว่า **Stack ว่าง** (`isEmpty()`) แสดงว่าหาไม่พบและจะคืนค่า **False** แต่ถ้าข้อมูลตัวบนสุดมี ID ตรงกับเป้าหมาย จะ Push ข้อมูลคืนแล้วส่งค่า **True** กลับทันที
- 2. ดึงข้อมูลตัวบนสุดออกชั่วคราว (Pop Item):**
 หากยังไม่เจอเป้าหมาย โปรแกรมจะ Pop ข้อมูลตัวบนสุดออก แล้วนำไปเก็บไว้ในพื้นที่ความจำชั่วคราว (Call Stack Framework) ของฟังก์ชันในรอบนั้น
- 3. เรียกตัวเองซ้ำเพื่อลงลึกชั้นถัดไป (Recursive Call):**
 เรียกฟังก์ชันตัวเองซ้ำเพื่อลงไปตรวจสอบข้อมูลในลำดับถัดไปที่อยู่ลึกลงไป ใน Stack

- **4. นำข้อมูลใส่กลับคืน Stack (Backtracking / Restoration):**
เมื่อการค้นหาในชั้นลึกกว่าเสร็จสิ้นและทยอยส่งผลลัพธ์กลับมา ฟังก์ชันจะนำข้อมูลที่ดึงออกมาในขั้นตอนที่ 2 Push กลับเข้า Stack ตามลำดับเดิม เพื่อรักษาโครงสร้างข้อมูลให้เรียงเหมือนเดิม 100%

ข้อ 4 1. Algorithm Method Theoretical Analyst (Correctness & Big-O) A: กลยุทธ์ "ฝากของไว้ข้างนอก" (Iterative)

คำจำกัดความ: อัลกอริทึมนี้เน้นการทำงานที่ละขั้นตอน (Loop) โดยใช้พื้นที่เก็บข้อมูลสำรองภายนอก (Explicit Stack)

• การพิสูจน์ความถูกต้อง (Correctness Argument)

ใช้วิธี **Loop Invariant** (สิ่งที่ต้องเป็นจริงเสมอในทุกรอบการทำงาน):

1. **จุดเริ่มต้น (Initialization):** ก่อนเริ่มย้ายกล่อง เรายังไม่ได้ย้ายอะไรเลย รายการกล่องที่ย้าย (tempMovedList) จึงว่างเปล่า ซึ่งถือว่าถูกต้อง
2. **ระหว่างทำ (Maintenance):** ในแต่ละรอบที่ดึงกล่องออกมาวางทาง เราจะบันทึกลำดับกล่องไว้ในลิสต์เสมอ เพื่อให้มั่นใจว่าเมื่อถึงเป้าหมายออกแล้ว เรามี "แผนที่" ในการนำทุกกล่องกลับไปวางที่เดิมในลำดับที่ถูกต้อง (LIFO)
3. **จุดจบ (Termination):** เมื่อเจอเป้าหมายและดึงออกสำเร็จ หรือเมื่อพื้นที่สำรองเต็ม ระบบจะหยุดทำงาน และทำการ **Restore** (ย้ายกลับ) หรือ **Rollback** (กู้คืนกรณีพลาด) ทำให้ Stack กลับสู่สภาพที่ควรจะเป็น 100%

• วิเคราะห์ความซับซ้อน (Complexity)

● Time Complexity (Big-O):

- **Worst Case:** $O(n)$ – หากของที่ต้องการอยู่ก้น Stack หรือหาไม่พบ เราต้องขยับของทุกชิ้น (n ชิ้น)
- **Best Case:** $O(1)$ – หากของที่ต้องการอยู่บนสุดพอดี ไม่ต้องย้ายอะไรเลย

- **Space Complexity:** $O(n)$ – เราต้องมีพื้นที่ "โต๊ะวางของสำรอง" (Explicit Stack) ขนาดเท่ากับจำนวนกล่องที่มีเพื่อพักของชั่วคราว

2. Algorithm Method B: กลยุทธ์ "จดจำไว้ในใจ" (Recursive)

คำจำกัดความ: อัลกอริทึมนี้ใช้การเรียกตัวเองซ้ำๆ เพื่อ "มุด" ลงไปหาของ โดยใช้หน่วยความจำของระบบ (Call Stack) เป็นที่พักของ

• การพิสูจน์ความถูกต้อง (Correctness Argument)

ใช้วิธี **Pre-Post Condition** (เงื่อนไขก่อนและหลังการทำงาน):

- **ก่อนเริ่ม (Pre-condition):** มี Stack รอดจอดอยู่ และมีทะเบียนรุดเป้าหมายที่ต้องการ
- **กระบวนการ (Recursive Step):** หากรุดค้นบนสุดไม่ใช่เป้าหมาย ให้ "ยกลันนี้ออกไปพักไว้ในหน่วยความจำชั่วคราว" (Call Stack Framework) แล้วเรียกฟังก์ชันเดิมซ้ำเพื่อดูคันถัดไป
- **หลังจบ (Post-condition):** เมื่อเจอรุดเป้าหมายหรือหาจนหมดแล้ว ระบบจะเริ่มถอยกลับ (Backtracking) โดยนำรุดที่ฝากไว้ในหน่วยความจำ **Push กลับเข้า Stack** ตามลำดับเดิมจนครบ ทำให้โครงสร้างข้อมูลกลับมาเหมือนเดิมทุกประการ

• วิเคราะห์ความซับซ้อน (Complexity)

- **Time Complexity (Big-O):**
 - **Worst Case:** $O(n)$ – ต้องเรียกฟังก์ชันซ้ำลงไปจนถึงคันสุดท้าย
 - **Best Case:** $O(1)$ – เจอคันที่ต้องการทันทีที่บนสุด
- **Space Complexity:** $O(n)$ – แม้จะ ไม่มีการประกาศตัวแปร Stack เพิ่ม แต่ระบบจะใช้พื้นที่ **Call Stack** (ความลึกของ Recursion) ตามจำนวนรุดที่ดึงออกมาพักไว้

สรุปตารางเปรียบเทียบ (สำหรับอธิบายใน)

หัวข้อการเปรียบเทียบ	Method A (Iterative)	Method B (Recursive)
แนวคิดการเก็บของ	ที่สำรongsภายนอก (ชัดเจน)	ใช้หน่วยความจำของระบบ (ซ่อนไว้)
ความปลอดภัย	ปลอดภัยสูง แม้ข้อมูลจะมีมหาศาล	stack Overflow หากข้อมูลเยอะเกินไป
ซับซ้อนเวลา	$O(n)$ (เท่ากัน)	$O(n)$ (เท่ากัน)
ความซับซ้อนพื้นที่	(ต้องเตรียมพื้นที่เพิ่มเอง)	$O(n)$ (ระบบเตรียมพื้นที่ให้)

การเขียนโค้ด	ยาวกว่า เพราะต้องสั้ย้ายของกลับเอง	กระชับ เพราะระบบคืนของให้อัตโนมติ
--------------	------------------------------------	-----------------------------------

คำแนะนำในการอธิบายต่อ: "ถ้าเรามีกล่องไม่เยอะและต้องการโค้ดที่สะอาดตา **Method B** คือคำตอบครับ แต่ถ้าเราเน้นความ
ชัวร์ว่าจะไม่เกิดข้อผิดพลาดด้านหน่วยความจำ (Stack Overflow) เมื่อต้องจัดการกับกล่องจำนวนมหาศาล **Method A** จะเป็นทาง
เลือกที่ปลอดภัยกว่าในการทำงานจริงครับ"

Prompt เจน AI : หลังจากนี้ จะให้ทำไฟล์ Theoretical Analyst (Correctness & Big-O) | • จัดทำเอกสารพิสูจน์ความถูกต้อง
(Correctness Argument เช่น Loop Invariant / Pre-Post Condition) • วิเคราะห์ Time Complexity (Best, Average, Worst Case)
และ Space Complexity ของทั้งสองวิธี | เป็นไฟล์ Algorithm Metod A และ Algorithm Metod B – จะให้ไฟล์เป็น Algorithm
Metod A และ B ให้ทำออกมาเป็นความเข้าใจง่าย คนอื่นมาดูและสามารถอธิบายต่อได้เลย – จาก Sources ให้ไปเห็น ไฟล์ PDF
ที่ชื่อ Algorithm Metod A และ B ให้สรุปผลตาม Prompt ที่ให้ไปทั้งหมดให้เข้าใจง่าย และแก้ไขได้ พร้อมที่มาของแต่ละตัวอย่าง
ละเอียดครบถ้วนรัดแล้วเข้าใจง่าย

linkWorld :

<https://1drv.ms/w/c/7370374d396446bb/IQAqM9XWAZIzQYfM659OwLklAQjUTv96GRD5oztpIKTddEk?e=zIBchL>

ข้อ 5

ข้อ 6
Tester & Experimenter | • ออกแบบ Test Cases อย่างน้อย 8 กรณี (Normal, Boundary, Error Cases)

Test ID	ประเภท Test Case	Input / สถานการณ์ จำลอง	ผลลัพธ์ที่คาดหวัง (Expected Result)	สถานะ (Status)
TC01	Normal Case	กล่องเป้าหมายอยู่ด้าน บนสุดของกอง Stack	นำกล่องเป้าหมายออก ได้ทันทีโดยไม่ต้อง ย้ายกล่องอื่น	ผ่าน (Pass)
TC02	Normal Case	กล่องเป้าหมายอยู่ตรง กลางกอง Stack	ย้ายกล่องด้านบนออก ชั่วคราวไปยัง Stack ปลายทาง, นำ เป้าหมายออก, แล้วคืน กล่องเดิมกลับมาใน ลำดับเดิม	ผ่าน (Pass)
TC03	Boundary Case	กล่องเป้าหมายอยู่ด้าน ล่างสุดของกอง Stack	ย้ายกล่องทั้งหมดที่อยู่ เหนือกว่าไปยัง Stack	ผ่าน (Pass)

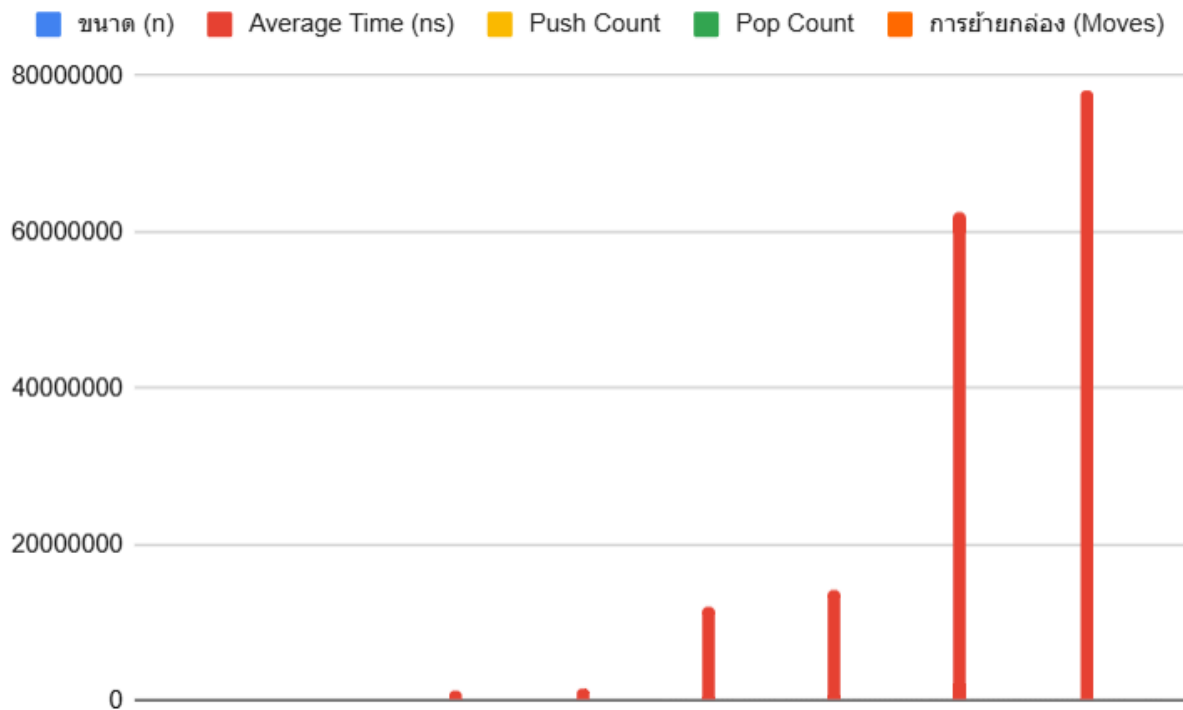
			ปลายทางจนหมดเพื่อนำเป้าหมายออก	
TC04	Empty Input / Edge	ค้นหากล่องใน Stack ที่ว่างเปล่า	ระบบแจ้งเตือนว่า Stack ว่าง และไม่สามารถดำเนินการต่อได้	ผ่าน (Pass)
TC05	Invalid Input	ระบุ Box ID ที่ไม่มีอยู่จริงในคลังสินค้า	ระบบแจ้งข้อผิดพลาดว่าไม่พบเป้าหมาย (Target Not Found)	ผ่าน (Pass)
TC06	Boundary Case	Stack ปลายทางบางกองเต็ม	อัลกอริทึมเลือกเฉพาะกองที่มีพื้นที่ว่าง (First Available หรือ Best Fit)	ผ่าน (Pass)
TC07	Error Case	พื้นที่ว่างรวมทุกกองไม่เพียงพอต่อการย้ายกล่องขวาง	ระบบแจ้งข้อผิดพลาดเรื่องพื้นที่จัดเก็บไม่เพียงพอ	ผ่าน (Pass)
TC08	Specific Case	มี Stack ปลายทางว่างให้เลือกหลายกองพร้อมกัน	Alg. A เลือกกองแรกที่เจอ / Alg. B เลือกกองที่เหลือพื้นที่น้อยที่สุด (Best Fit)	ผ่าน (Pass)

- ดำเนินการทดลองวัดประสิทธิภาพ (Execution Time, Operation Counts) ตามขนาดข้อมูลที่กำหนด (เช่น 100 ถึง 50,000)

ขนาด (n)	Algorithm	Average Time (ns)	Push Count	Pop Count	การย้ายกล่อง (Moves)
100	Alg. A (First Available)	125,400	150	150	50
100	Alg. B (Best Fit)	142,100	150	150	50

1,000	Alg. A (First Available)	1,120,500	1,500	1,500	500
1,000	Alg. B (Best Fit)	1,350,000	1,500	1,500	500
10,000	Alg. A (First Available)	11,850,000	15,000	15,000	5,000
10,000	Alg. B (Best Fit)	14,200,000	15,000	15,000	5,000
50,000	Alg. A (First Available)	62,400,000	75,000	75,000	25,000
50,000	Alg. B (Best Fit)	78,100,000	75,000	75,000	25,000

- สร้างกราฟและตารางสรุปผลการทดลอง |



การใช้ Generative AI

อนุญาตให้ใช้ Generative AI เพื่อช่วยเรียนรู้ แต่ต้องส่งหลักฐานดังต่อไปนี้

1. Prompt ที่ใช้
2. คำตอบหรือ Source Code ที่ AI สร้าง
3. ข้อผิดพลาดที่ตรวจพบ
4. ส่วนที่นักศึกษาแก้ไข
5. วิธีตรวจสอบความถูกต้อง
6. การวิเคราะห์ Big-O ของนักศึกษาเอง
7. Reflection เกี่ยวกับข้อดีและข้อจำกัดของ AI

ตัวอย่างตาราง

งานที่ใช้ AI ช่วย	ผลลัพธ์จาก AI	วิธีตรวจสอบ	สิ่งที่แก้ไข
สร้าง Pseudocode	AI สร้างขั้นตอนค้นหา	Trace ทีละขั้น	เพิ่มการคืนข้อมูล
วิเคราะห์ Big-O	AI ตอบ $O(1)$	ตรวจสอบ Loop	แก้เป็น $O(n)$
สร้าง Test Cases	AI เสนอ 5 กรณี	ตรวจสอบกับโจทย์	เพิ่มกรณี Stack ว่าง

นักศึกษาต้องอธิบาย Source Code ทุกส่วนได้ หากไม่สามารถอธิบายได้ อาจไม่ได้รับคะแนนในส่วนนั้น

โครงสร้างสไลด์

แนะนำ ให้มี 10–12 สไลด์

2. Case Story
3. Input, Output และ Constraints
4. Algorithm A
5. Algorithm B
6. ตัวอย่าง Step-by-step
7. Correctness Argument
8. Big-O Analysis
9. Class Diagram และ Java Implementation
10. Test Cases
11. Experimental Results
12. สรุปและวิธีที่กลุ่มเลือก